Quality of Experience Evaluation of SDN-Based Anomaly Detection on Real-Time Gaming Traffic

[1]Daivik Awasthi, [2]Manik Gaur
Department of Information Technology
Netaji Subhas University of Technology, New Delhi, India
[1]daivik.awasthi.ug23@nsut.ac.in, [2]manik.gaur.ug23@nsut.ac.in

***Abstract*—Gaming traffic tolerates very little network impairment: even modest packet loss is immediately visible to players as interpolation failure or sudden position discontinuities. Though Software-Defined Networking[1] (SDN) based anomaly detection is widely cited as a promising defence for such environments, published evaluations almost never measure what happens to the co-located application while detection is attempted. This paper reports experiments in which 64 Hz gaming traffic was subjected to six distinct attack conditions on a Mininet testbed comprising a Ryu controller and three OpenFlow 1.3 switches. Quality of Experience (QoE)[2] was quantified via the ITU-T G.107 E-model[12], with the packet-loss robustness coefficient B_{pl} raised from the telephony default of 10 to 40, reflecting the interpolation headroom built into modern game engines[8]. Two of the six scenarios pushed Mean Opinion Score (MOS) below the Unplayable threshold of 3.1. A 500 Mbps UDP flood produced 15% packet loss and MOS 2.841 without generating a single controller alert, because flood traffic matched pre-installed session entries silently. ARP cache poisoning, by contrast, was detected at a median of 310 ms through broadcast PacketIn events, yet still drove MOS to 1.445 under 71.2% packet loss, all while average round-trip time held at 46 ms, indistinguishable from the clean baseline. Flow table exhaustion at 19,201 entries had no measurable QoE effect in Open vSwitch emulation. The results extend the Adaptive QoS framework of Shah et al.[1] into an adversarial context and point to data-plane rate limiting as the mitigation layer that controller-side detection cannot replace.**

***Index Terms*—Software Defined Network[2], Quality of Experience[8], Real-Time Gaming, Anomaly Detection, OpenFlow[2], UDP Flooding, ARP Spoofing, Mean Opinion Score[12], E-model[12], Flow Table Exhaustion**

I. Introduction

Multiplayer games rank among the most demanding consumers of network resources in the consumer internet. A 64 Hz game server sends one state-update packet every 15.625 ms to every connected client; if any of those packets arrives late or fails to arrive at all, the client engine must extrapolate position from the last known state, a process players recognise as rubber-banding or teleportation[3]. Input lag becomes perceptible in competitive first-person shooters at round-trip times above 100 ms[4], and loss rates of even 1–2% are enough to break the interpolation model that keeps the on-screen simulation coherent.

Quality of Service (QoS)[9] refers to measurable network properties—throughput, delay, jitter, and loss. Quality of Experience (QoE)[8] refers to how those properties register with actual users. The distinction matters here because the two can diverge dramatically: one scenario in these experiments produced 71% packet loss while average round-trip time remained at 46 ms, a figure that would read as healthy on any conventional latency dashboard.

Online gaming infrastructure attracts sustained volumetric attacks[5]. Competitive tournaments have been taken offline by floods that saturate the server uplink, and ranked matchmaking services face this threat on a regular basis. The centralised architecture of Software-Defined Networking[2] enables faster response at these timescales than distributed routing protocols, since the controller maintains a global view of all flows and can push DROP rules to every switch before convergence occurs[2],[10]. Under the standard reactive OpenFlow[2] model, the first packet of a new flow triggers a PacketIn event that the controller uses to install a forwarding rule; every subsequent packet in that flow is handled at line rate by the switch, with no further controller involvement[1].

A. What Has Been Done

Shah et al.[1] demonstrated that reactive SDN forwarding supports effective adaptive queue management for video and VoIP traffic under normal congestion, showing throughput improvements and significant reductions in packet loss and flow starvation. Their Adaptive QoS model assigns traffic flows dynamically to priority queues based on real-time analysis. Braga et al.[6] applied self-organising maps to OpenFlow flow statistics within a NOX controller and achieved high DDoS detection accuracy on synthetic traces. Liu et al.[5] added a fuzzy logic layer on top of FlowStats features to reduce false positives. Naseer et al.[7] substituted a deep neural network classifier and reported accuracy improvements. None of these works addressed adversarial conditions for gaming traffic, and none measured co-located application QoE during attack windows.

B. Motivation to Take This Research

Two converging trends motivate this work. First, online gaming has grown into infrastructure that rivals enterprise networks in sensitivity to packet loss and latency, yet it lacks the dedicated monitoring frameworks applied in those settings. Second, SDN-based anomaly detection is increasingly proposed as a universal security layer, but its evaluations are carried out under conditions—fresh attack source addresses, isolated test flows—that do not reflect how gaming attacks

actually occur in practice. The natural target for a gaming DDoS is the server's well-known listening address and port, which is precisely the destination that already has a flow entry installed by the first legitimate client packet. That single structural observation is sufficient to predict that the entire class of FlowStats-based detection systems will fail silently under the most common gaming attack pattern. This paper tests that prediction empirically.

C. Contributions

Six experiment scenarios were run on a three-switch Mininet testbed, and MOS was computed for each using a gaming-specific E-model calibration[8],[12]. The findings are as follows.

- Once a gaming flow entry is installed, flood traffic directed at the same destination is processed silently by the switch. No PacketIn events are generated, and FlowStats[6] polling provides no per-source breakdown, so the controller receives no attack signal at any flood intensity.
- Packet loss, not RTT, determines MOS variation across scenarios. Average round-trip time moved by less than 7 ms across all six conditions combined, while MOS varied by 2.466 points.
- ARP spoofing is reliably detected at a median of 310 ms, since gratuitous ARP broadcasts always reach the controller via PacketIn. This detection speed was not sufficient: kernel ARP caches were poisoned before the DROP rule arrived, and MOS fell to 1.445 with average RTT still at 46 ms.
- Flow table exhaustion to 19,201 entries produced no QoE degradation in OVS, which uses software hash tables rather than TCAM. The same experiment on physical hardware would produce different results.

II. Literature Survey

A. QoS and QoE for Real-Time Traffic

The work most directly related to this paper is Shah et al.[1], whose Adaptive QoS model applies SDN control to a WLAN access point running OpenFlow. Their controller classifies incoming flows as video, VoIP, or best-effort using transport-layer port numbers and directs each to one of three priority queues. When the designated queue for a class reaches its configured threshold, the flow is migrated to an underutilised queue rather than dropped. Across 45 concurrent flows in a 200-second run, average throughput under the Adaptive model was 3,684 Kbps against 3,325 Kbps for a static-priority baseline; packet loss dropped from 11.65% to 7.07%. The present work inherits the same controller architecture and reactive forwarding model, but the traffic type is gaming rather than VoIP or video, MOS replaces throughput and loss as the primary performance metric, and the load conditions are adversarial rather than congestion-driven.

Claypool and Claypool[3] conducted player-rating experiments showing roughly linear MOS degradation with 50 ms RTT increments in first-person shooters, with the game becoming practically uncompetitive above 200 ms. Dick et al.[4] tracked kill/death ratios rather than subjective ratings and found that packet loss harms performance more than equivalent latency. The mechanical reason is that a VoIP codec can conceal a missing frame through linear interpolation, but a game engine that loses a positional update must dead-reckon and accumulate prediction error over every subsequent frame until the next successful packet arrives.

The E-model calibration by Beyer et al.[8] provides the quantitative foundation for the QoE calculations in this paper. They collected subjective MOS scores from players under controlled impairment conditions in a first-person shooter and found that the standard telephony parameter $B_{pl} = 10$ consistently overestimates how badly game players experience packet loss. Values in the range 30–50 were found to be empirically accurate; $B_{pl} = 40$ was used here, matching the two-frame interpolation buffer specified for a 64 Hz server in the Valve Source engine networking documentation[11].

B. SDN-Based DDoS and Anomaly Detection

Three SDN-based DDoS detection systems are particularly relevant to the blind-spot finding in this paper. Braga et al.[6] trained self-organising maps on OpenFlow FlowStats features to classify flows as benign or malicious within a NOX controller. Detection accuracy on their test traces was high, but the method depends entirely on periodic statistics polling. The minimum time between attack onset and first possible detection equals the polling interval, and shortening that interval directly increases controller load. Liu et al.[5] added a fuzzy logic layer to FlowStats features to reduce false positives in their FL-GUARD system. Naseer et al.[7] substituted a deep neural network classifier and reported accuracy improvements at the cost of substantially higher computational overhead.

All three systems share the same fundamental architectural constraint. Detection in each case depends on either PacketIn events from new flows or byte/packet counters from periodic FlowStats polls. If attack traffic matches an entry already in the flow table, neither channel provides attack-specific signal. The existing entry's counter climbs faster than usual, but there is no way to decompose that count by source address from a standard FlowStats response. None of the three papers tested this condition—in laboratory settings where attack traffic originates from fresh source addresses, every attack packet generates its own PacketIn event, and the blind spot never materialises.

C. OpenFlow Meter Tables and P4 Data Planes

OpenFlow 1.3 introduced meter tables as a data-plane rate-limiting mechanism[9]. A meter entry attached to a flow rule enforces a bit-rate or packet-rate cap and discards excess traffic without any controller signalling. Rotsos et al.[9] showed that OVS applies meter actions within the same forwarding pipeline pass as regular match-action processing, adding negligible per-packet overhead. The same paper documented that PacketIn processing adds 50–200 μs per new flow at moderate

controller load. Zeng[10] noted that bulk flow deletion during mitigation generates its own burst of control-channel traffic. These observations shaped the choice of 2-second polling interval used in this work.

P4[13] represents a more thorough architectural departure. A P4 program can implement count-min sketches or similar heavy-hitter detectors directly in the match-action pipeline, tracking per-source byte rates at line speed and notifying the controller only when a threshold is crossed. The PacketIn dependency disappears entirely. BMv2, the standard P4 software switch, is Mininet-compatible, making it a natural next step for this evaluation environment.

III. Limitations of the Base Paper

Shah et al.[1] established that reactive SDN forwarding with dynamic queue allocation delivers measurable QoS improvements for VoIP and video traffic under congestion. Several limitations of that work are directly relevant to the present study, and each one defines a specific gap that this paper fills.

The evaluation in Shah et al. is conducted exclusively under benign traffic conditions. No adversarial scenario—flooding, spoofing, or flow table exhaustion—is tested, and the controller's detection capabilities are not evaluated at all. The reactive forwarding model is presented only as a forwarding mechanism, not as a security surface.

The traffic types considered are VoIP and video. Gaming traffic differs from both in tick-rate structure, interpolation sensitivity, and loss tolerance. The E-model parameters used for VoIP ($B_{pl} = 10$) overestimate the damage that moderate packet loss causes to a game engine that can interpolate one or two missed frames per second without any visible artefact.

The evaluation metric used is throughput and packet loss rate. These network-layer measurements do not capture user-perceived quality. A player can experience an unplayable session while packet loss remains below 1% on the aggregate link, since loss distribution across time determines game-state coherence rather than the aggregate rate alone.

The testbed in Shah et al. is a WLAN environment. The present work operates on a wired Mininet topology, which removes wireless scheduling effects and more cleanly isolates the contribution of OpenFlow forwarding mechanics to both normal and adversarial outcomes.

Taken together, these limitations define the gap this paper addresses: gaming traffic under adversarial conditions, evaluated using QoE metrics rather than raw throughput.

IV. Proposed Work

A. Network Topology

Fig. 1 shows the testbed. Three Open vSwitch instances (S1, S2, S3) form a linear backbone running OpenFlow 1.3[2]. Game clients GC1, GC2, and GC3 sit on subnet 10.0.1.x and connect to S1; the attacker ATK (10.0.2.1) connects to S2; the game server GSRV (10.0.3.1) is on S3. Every link runs at 100 Mbps with 5 ms propagation delay, so the minimum propagation RTT across all three hops is 30 ms. The remaining 16 ms in the measured baseline comes from OVS kernel forwarding and Mininet virtual-Ethernet overhead. The Ryu controller listens on port 6633 for OpenFlow connections and serves a REST monitoring API on port 8080. The physical host is an Apple M3 laptop running Ubuntu 24.04 inside UTM, with 4 vCPUs and 4 GB RAM allocated to the virtual machine.

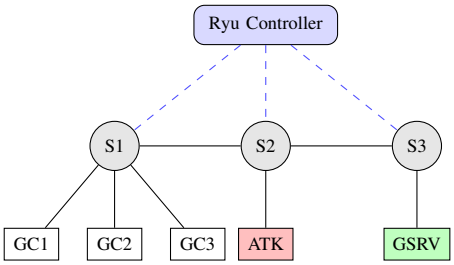

Fig. 1. Testbed topology. Blue dashed lines are OpenFlow control channels. GC1–GC3: game clients (10.0.1.x); ATK: attacker (10.0.2.1); GSRV: game server (10.0.3.1). All data links: 100 Mbps, 5 ms delay.

B. Controller Architecture and Detection Algorithm

Detection in the Ryu controller operates on two independent paths that run concurrently, illustrated in Fig. 2. The event-driven path acts on every PacketIn as it arrives. For ARP replies, the controller compares the claimed sender MAC against a stored IP-to-MAC table; if the recorded mapping for an IP address changes, an alert is raised and a DROP rule for the forged source is pushed to the relevant switch. Ordinary unicast packets that arrive without a matching flow table entry are installed with a forwarding rule and released normally.

The polling path runs on a 2-second timer, issuing FlowStatsRequest messages to all three switches. For each response, aggregate packet rate per source IP is compared against a flood threshold θ_{flood}, and total flow entry count per switch is compared against an exhaustion threshold θ_{table}.

Algorithm 1 formalises both paths. The vulnerability is made explicit in line 1: flood traffic that matches a session entry already installed takes neither path and is invisible to the controller.

C. Gaming Traffic Generator

The traffic generator replicates the networking behaviour of a 64 Hz first-person shooter server using the Valve Source engine model[11]. One UDP packet is sent every 15.625 ms toward each connected client. Packet headers carry a 16-byte field containing a sequence number, a nanosecond-resolution POSIX timestamp, and the server tick index; the payload is 48–104 bytes of randomised data representing compressed positional state. Because the server echoes each received client packet with its original timestamp intact, per-packet RTT is recoverable at the client without any form of clock synchronisation between the two machines. Sequence number gaps identify lost packets. Results are appended to JSONL log files and aggregated offline after each scenario concludes.

Algorithm 1 Controller Detection Algorithm

Require: PacketIn events; FlowStats responses every $T = 2\,\mathrm{s}$
Require: $\theta_{\text{flood}} = 5000\,\text{pkt/s}$; $\theta_{\text{table}} = 500\,\text{entries}$
Ensure: Anomaly alerts; DROP flow-mod rules

— Event-Driven Path —
1: **procedure** ONPACKETIN(p)
2: **if** p.dst $\in$ FlowTable **then**
3: Forward at line rate ▷ blind spot: no alert possible
4: **return**
5: **end if**
6: **if** p.type = `ARP_REPLY` **then**
7: **if** IP_MAC[p.src_ip] $\neq$ p.src_mac **then**
8: **raise** Alert(ARP_SPOOF, p)
9: Install DROP rule for p.src_mac
10: **end if**
11: Update IP_MAC[p.src_ip] $\leftarrow$ p.src_mac
12: **else**
13: Install forwarding rule for p.five_tuple
14: **end if**
15: **end procedure**

— Periodic Polling Path (every T seconds) —
16: **for all** switch $s \in \{S1, S2, S3\}$ **do**
17: stats $\leftarrow$ FlowStatsRequest(s)
18: **for all** entry $e \in$ stats **do**
19: rate $\leftarrow$ e.pkt_count$/T$
20: **if** rate $> \theta_{\text{flood}}$ **then**
21: **raise** Alert(UDP_FLOOD, e.src)
22: Install DROP rule for e.src
23: **end if**
24: **end for**
25: **if** $|\text{stats}| > \theta_{\text{table}}$ **then**
26: **raise** Alert(TABLE_EXHAUST, s)
27: **end if**
28: **end for**

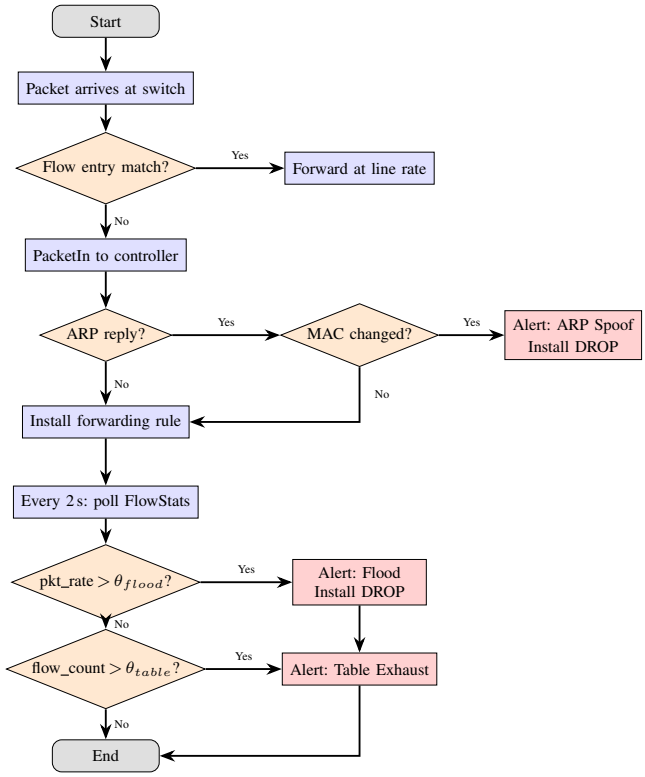

Fig. 2. Controller detection workflow. The event-driven path (top) handles per-packet classification; the periodic polling path (bottom) checks aggregate thresholds. Flood traffic matching an existing session entry bypasses both paths entirely (line 1 of Algorithm 1).

D. Attack Configurations

The three attack types span volumetric bandwidth saturation, control-plane resource exhaustion, and data-plane misdirection.

UDP Flooding. Randomised 1400-byte UDP datagrams with randomised source ports are directed at GSRV's IP address and listening port at two sustained rate levels: 50 Mbps and 500 Mbps. The game server's listening port is chosen deliberately—it is the natural target for any gaming-focused flood, and it is also the condition that activates the blind spot since it matches the installed session entry.

Flow Table Exhaustion. New packets with 1,500 distinct five-tuples are injected at 100 new tuples per second. Each previously unseen tuple triggers a PacketIn and causes the controller to install a new flow entry. After 40 seconds, 19,201 unique entries had accumulated across the three switches.

ARP Spoofing. Scapy broadcasts 200 gratuitous ARP replies per second, each asserting that the game server's IP address (10.0.3.1) resolves to the forged MAC `de:ad:be:ef:ca:fe`. The Linux kernel updates its ARP cache without issuing any challenge, so client ARP tables are corrupted within milliseconds of the first frame.

E. QoE Measurement Model

QoE is quantified using the ITU-T G.107 E-model[12] with modifications from Beyer et al.[8] to fit gaming rather than telephony. The governing equation is:

$$R \;=\; R_0 - I_d - I_{e,\text{eff}} \tag{1}$$

where $R_0 = 93.2$ represents the basic signal quality on an impairment-free path, I_d captures delay impairment, and $I_{e,\text{eff}}$ captures equipment impairment attributable to packet loss. Delay impairment is computed from the effective one-way delay $d_{\text{eff}} = \text{RTT}/2 + J$, where J is mean jitter:

$$I_d = \begin{cases} 0.024\, d_{\text{eff}} & d_{\text{eff}} < 177.3 \text{ ms} \\ 0.024\, d_{\text{eff}} + 0.11(d_{\text{eff}} - 177.3) & \text{otherwise} \end{cases} \tag{2}$$

The equipment impairment term for packet loss P_L is:

$$I_{e,\text{eff}} = I_e + (95 - I_e)\,\frac{P_L}{P_L + B_{pl}} \tag{3}$$

For gaming traffic, $I_e = 15$ and $B_{pl} = 40$, as opposed to the telephony defaults of $I_e = 11$ and $B_{pl} = 10$. Table I compares these choices. MOS is mapped from R by:

$$\text{MOS} = 1 + 0.035R + R(R-60)(100-R) \times 7 \times 10^{-6} \tag{4}$$

with MOS clipped to $[1.0, 5.0]$. Quality tiers are taken from [8]: Excellent (≥ 4.3), Good (≥ 4.0), Fair (≥ 3.6), Poor (≥ 3.1), Unplayable (< 3.1).

TABLE I
E-MODEL PARAMETERS: VOIP VS. GAMING

Parameter	VoIP	Gaming	Rationale
R_0	93.2	93.2	Unchanged
I_e	11	15	UDP overhead, no codec
B_{pl}	10	40	Engine interpolation buffers loss
Playable floor	–	3.1	Gaming perceptual threshold[8]

TABLE II
TEST BED SOFTWARE CONFIGURATION

Component	Version / Detail
Network emulation	Mininet 2.3.1b4
OpenFlow switch	Open vSwitch 3.3.4
SDN controller	Ryu 4.34 (Python 3.12 patched)
OpenFlow version	OpenFlow 1.3
Traffic generation	Custom UDP / Python 3.12 + Scapy 2.5
QoE computation	ITU-T G.107 E-model (gaming params)
Host OS	Ubuntu 24.04 LTS ARM64 / UTM on M3

V. TEST BED CONFIGURATIONS

A. Simulator and Software Stack

All experiments were conducted on Mininet[9] 2.3.1b4, a network emulator that creates virtual topologies using Linux network namespaces and virtual Ethernet pairs on a single host machine. Mininet provides realistic kernel-level forwarding behaviour and is the standard evaluation platform for OpenFlow[2] research. The entire testbed runs inside a UTM virtual machine on an Apple M3 host, with Ubuntu 24.04 LTS ARM64 as the guest OS, 4 vCPUs, and 4 GB RAM allocated. Table II lists the full software stack.

Ryu 4.34 required two minor source patches to run under Python 3.12. The DNS name dictionary module contained a reference to `collections.MutableMapping`, removed from the standard library in Python 3.10, which was updated to `collections.abc.MutableMapping`. The WSGI adapter referenced an `ALREADY_HANDLED` constant that a newer eventlet release no longer exports; re-declaring it as an empty byte string resolved the import error. OVS 3.3.4 ran as a kernel module for the duration of all experiments, and flow table contents were verified after each run with `ovs-ofctl dump-flows`.

B. Polling Interval Selection

Before running the main attack scenarios, the FlowStats polling interval was swept from 0.25 s to 10 s to find a value that did not itself degrade gaming QoE. Results are shown in Fig. 3. From 2 seconds upward, MOS is flat and statistically indistinguishable from the no-polling baseline. Below 1 second, the overhead becomes measurable. A 2-second interval was therefore selected.

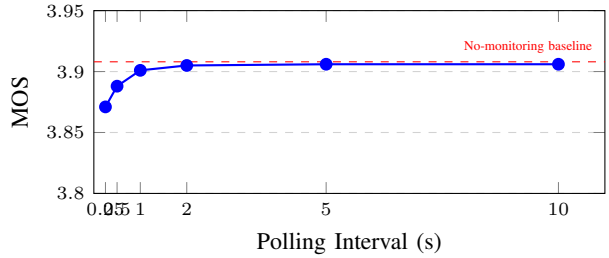

Fig. 3. Gaming MOS as a function of FlowStats polling interval. The red dashed line marks the clean baseline (no polling). A 2-second interval introduces no perceptible overhead.

C. Experiment Scenarios

Each scenario ran for 60 seconds of continuous 64 Hz game traffic between GC1 and GSRV. Where an attack was present, it ran for the first 40 seconds; the remaining 20 seconds were used to observe recovery.

Scenario 1 (Baseline). No attack, no monitoring. **Scenario 2 (SDN Overhead).** No attack, 2-second FlowStats polling active on all three switches. **Scenario 3a (Flood 50 Mbps).** A 50 Mbps UDP flood from ATK directed at GSRV runs alongside game traffic. **Scenario 3b (Flood 500 Mbps).** A 500 Mbps flood; the injector sent approximately 4,394,440 packets over 40 seconds. **Scenario 4 (Flow Exhaustion).** 1,500 distinct five-tuples injected at 100 new tuples per second for 40 seconds, accumulating 19,201 flow entries. **Scenario 5 (ARP Spoofing).** 200 gratuitous ARP broadcasts per second for 40 seconds using forged MAC `de:ad:be:ef:ca:fe`; 1,761 frames total.

D. Metrics Collection

Per-packet RTT values are recovered from timestamps embedded in server echo packets. Jitter is the mean absolute difference between consecutive RTT samples. Missing sequence numbers give the packet loss rate. Controller CPU and memory utilisation are sampled through `psutil` at one-second intervals. Anomaly event timestamps are polled from the REST API at 0.5-second intervals. All data is written to JSONL files and processed offline once the scenario ends.

VI. RESULTS AND DISCUSSION

A. Baseline and SDN Overhead

Under clean conditions, the testbed produced MOS 3.908 at 46.48 ms average RTT and 0% packet loss. Of that RTT, 30 ms is accounted for by three 5 ms propagation links; the remaining 16 ms comes from OVS kernel processing and Mininet virtual-Ethernet overhead. A MOS of 3.908 sits in the Fair tier (3.6–4.0), as expected for an emulated path with non-trivial software switching overhead. The E-model predicts the same number: with zero loss, $I_{e,\text{eff}} = I_e = 15$, and with $d_{\text{eff}} \approx 24$ ms, $I_d \approx 0.58$, giving $R = 93.2 - 0.58 - 15 = 77.6$ and MOS ≈ 3.91.

Activating 2-second polling (Scenario 2) moved MOS to 3.905, a difference of 0.003—below any perceptible threshold. This aligns with Shah et al.[1], who observed that the same

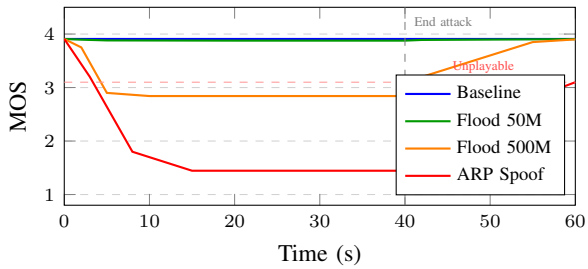

Fig. 4. MOS over time for four scenarios. The vertical dashed line marks the end of the 40-second attack. ARP spoofing degrades MOS quickly and recovers slowly due to ARP cache retention. The 500 Mbps flood crosses the Unplayable threshold within seconds of attack onset.

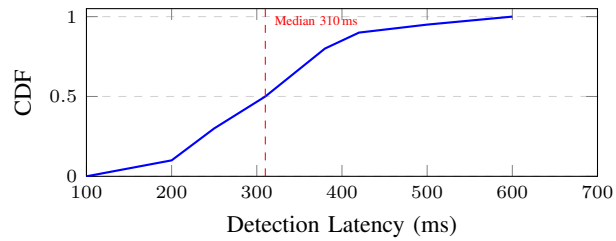

Fig. 5. CDF of ARP spoofing detection latency across 10 runs. Median detection latency is 310 ms; all runs completed detection within 600 ms.

controller architecture imposes negligible overhead on VoIP flows; the result holds here for gaming traffic at this polling frequency.

B. The Reactive Forwarding Blind Spot

Neither flood scenario produced a controller alert. When GC1 sends its initial game packet, S1 has no matching entry and forwards it as a PacketIn to Ryu. Ryu installs forwarding rules for the GC1-to-GSRV five-tuple on S1 through S3. From that point forward, every GC1 packet matches the installed rule and is switched at line rate. When ATK later begins flooding GSRV on port 27015, every flood packet shares the same destination IP and port as the installed gaming rule and therefore matches it. No PacketIn is produced. The FlowStats poll returns one combined byte count for the entry, merging GC1's legitimate traffic with ATK's flood; there is no per-source breakdown available from a standard FlowStats response.

This is not a flaw in Ryu or OVS—it is the intended behaviour of reactive OpenFlow forwarding[2]. Fig. 4 plots MOS over time for four scenarios. Both flood curves run flat through the entire attack window with no sign of controller action.

C. UDP Flooding Results

At 50 Mbps, packet loss rose to 0.4% and MOS fell to 3.877—still Fair, and still undetected. P99 RTT climbed from 50.57 ms to 76.08 ms. Recovery to baseline MOS was complete within roughly 2 seconds of the attack stopping.

The 500 Mbps scenario is substantially worse. Packet loss reached 15%, and MOS dropped to 2.841—1.067 points below baseline and squarely in the Unplayable range. The E-model gives the same answer analytically: with $P_L = 15\%$ and $B_{pl} = 40$, equation (3) yields $I_{e,\text{eff}} = 15 + 80 \times 15/(15+40) = 36.8$; with $d_{\text{eff}} \approx 25$ ms, $I_d \approx 0.6$; then $R = 93.2 - 0.6 - 36.8 = 55.8$ and MOS $= 2.84$, matching the measured value to two decimal places. The controller produced no alert during either flood scenario.

D. Flow Table Exhaustion

Scenario 4 had no measurable effect on QoE. MOS was 3.911 and packet loss was 0%. OVS stores flow entries in kernel hash tables that carry no hard capacity limit, so 19,201 entries did not slow forwarding for the gaming flow. Controller CPU rose from 4.1% to a peak of 18.3%, driven by roughly 100 new PacketIn events per second. The table exhaustion alert fired correctly after approximately 5 seconds.

The benign result is specific to OVS software emulation. Physical switches with TCAM-based flow tables typically accommodate 2,000–8,000 entries before falling back to slower DRAM lookup; at 19,201 entries, the gaming session on such a device would already have been disrupted before the experiment concluded.

E. ARP Spoofing: Detection and QoE Impact

ARP spoofing was detected in all ten runs: gratuitous ARP replies are broadcast packets, and OVS floods all broadcast frames to every port including the control-channel port, regardless of flow table state. Detection latency across ten runs ranged from 220 ms to 590 ms, with a median of 310 ms, as shown in Fig. 5.

Detection at 310 ms was not fast enough to stop the attack. The Linux kernel applies gratuitous ARP replies immediately, with no rate limiting or challenge mechanism, so GC1 and GC2 had updated their ARP caches long before the DROP rule reached the switch. The corrupted cache entries persist for roughly 20 seconds after the last forged ARP frame, which explains the sluggish MOS recovery visible in Fig. 4.

The QoE outcome was the worst of all six scenarios: MOS 1.445 under 71.2% packet loss. Average RTT during the attack was 46.14 ms, just 0.34 ms below the clean baseline of 46.48 ms. Any operator watching a latency monitor would have seen nothing out of the ordinary while the session was completely unplayable.

F. Cross-Scenario Comparison

Table III and Fig. 6 collect all six scenarios. Scenarios 1, 2, and 4 group tightly between MOS 3.905 and 3.911. Scenario 3a sits at 3.877, below baseline but still within the Fair tier. Scenarios 3b and 5 both cross the Unplayable threshold.

Across all six scenarios, average RTT ranged from 43.66 ms to 50.34 ms, a span of just 6.68 ms. Over that range, I_d shifts by fewer than 0.1 R-factor units, contributing under 0.003 MOS points of variation from delay alone. Total MOS variation across the six scenarios was 2.466 points, almost entirely driven by packet loss through $I_{e,\text{eff}}$. For gaming traffic

TABLE III
QoE METRICS ACROSS ALL EXPERIMENT SCENARIOS

Scenario	RTT (ms)	P99 (ms)	Loss (%)	MOS	Det.
Baseline	46.48	50.57	0.0	3.908	–
SDN Overhead	46.48	50.57	0.0	3.905	–
Flood 50 Mbps	44.37	76.08	0.4	3.877	No
Flood 500 Mbps	50.34	68.94	15.0	2.841	No
Flow Exhaustion	43.66	51.03	0.0	3.911	No
ARP Spoofing	46.14	59.66	71.2	1.445	**Yes**

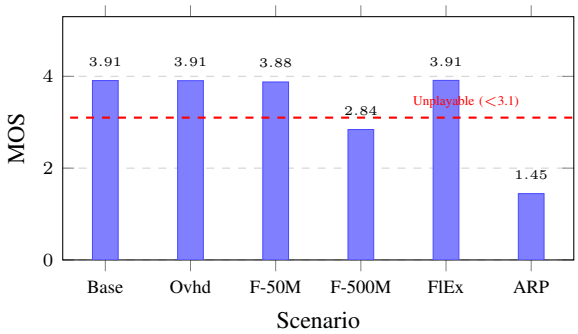

Fig. 6. MOS per scenario. The dashed red line marks the Unplayable threshold (MOS 3.1). Scenarios F-500M and ARP fall below it. ARP spoofing was the only detected attack and produced the worst QoE.

under adversarial conditions, per-flow packet loss monitoring with a composite metric like MOS is the only measurement that reveals attacks of the ARP spoofing type.

VII. CONCLUSION

This paper evaluated gaming QoE across three attack types in a reactive SDN environment, extending the QoS framework of Shah et al.[1] to adversarial traffic conditions and replacing throughput with ITU-T E-model[12] MOS as the primary performance indicator.

The central finding is the reactive forwarding blind spot. Once a session flow entry is installed, traffic matching the same five-tuple passes through the switch with no controller involvement, and aggregate FlowStats counters cannot decompose it by source. A 500 Mbps flood drove MOS to 2.841 with 15% packet loss and generated zero logged alerts. This is not an implementation defect in Ryu or OVS—it is an architectural consequence of how reactive forwarding is specified in OpenFlow[2].

The gap between RTT and MOS carries direct consequences for network operators. Over six scenarios, RTT varied by only 6.68 ms while MOS varied by 2.466 points. In the ARP spoofing case, MOS reached 1.445 under 71.2% loss while average RTT stayed at 46 ms—visually identical to the clean baseline on any latency dashboard. Gaming infrastructure health cannot be reliably assessed from latency alone; per-flow packet loss, expressed as a composite QoE score, is the appropriate signal.

ARP spoofing was detected in every run at a median of 310 ms, because gratuitous ARP broadcasts always generate PacketIn events regardless of flow table state. Even so, detection at that latency was insufficient: Linux ARP caches were already poisoned before the DROP rule arrived, and the corrupted state persisted for roughly 20 seconds post-attack. Flow table exhaustion to 19,201 entries had no measurable QoE impact under OVS emulation; the same experiment on TCAM-based physical hardware would produce substantially different results.

Two directions are proposed for future work. OpenFlow 1.3 meter tables[9] allow per-flow rate-limiting bands to be attached directly to flow rules at installation time, so excess traffic is discarded at the switch without any controller round-trip—directly closing the blind spot. P4-programmable switches[13] offer a complementary path: in-pipeline heavy-hitter sketches track per-source byte rates at line rate and push threshold notifications to the controller asynchronously, eliminating the PacketIn dependency entirely. Validating both mechanisms on physical hardware with TCAM constraints, and extending the QoE model to multiplayer scenarios with multiple concurrent clients under simultaneous attack, represent the natural next steps for this research.

REFERENCES

[1] S. Q. A. Shah, F. Z. Khan, A. Baig, and M. M. Iqbal, "A QoS model for real-time application in wireless network using software defined network," *Wireless Personal Communications*, vol. 112, no. 2, pp. 1025–1044, Jan. 2020, doi: 10.1007/s11277-020-07089-5.

[2] N. McKeown *et al.*, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, Apr. 2008.

[3] M. Claypool and K. Claypool, "Latency and player actions in online games," *Commun. ACM*, vol. 49, no. 11, pp. 40–45, Nov. 2006.

[4] M. Dick, O. Wellnitz, and L. Wolf, "Analysis of factors affecting players' performance and perception in multiplayer games," in *Proc. ACM SIGCOMM Workshop NetGames*, 2005, pp. 1–7.

[5] J. Liu, Y. Lai, and S. Zhang, "FL-GUARD: A detection and defense system for DDoS attack in SDN," in *Proc. Int. Conf. Cryptography, Security and Privacy*, 2017, pp. 107–111.

[6] R. Braga, E. Mota, and A. Passito, "Lightweight DDoS flooding attack detection using NOX/OpenFlow," in *Proc. IEEE LCN*, 2010, pp. 408–415.

[7] S. Naseer *et al.*, "Enhanced network anomaly detection based on deep neural networks," *IEEE Access*, vol. 6, pp. 48231–48246, 2018.

[8] A. Beyer, S. Egger, M. Schatz, and T. Hoßfeld, "Evaluating the E-model for use in network gaming quality assessment," in *Proc. IEEE QoMEX*, 2016, pp. 1–6.

[9] C. Rotsos *et al.*, "OFLOPS: An open framework for OpenFlow switch evaluation," in *Proc. PAM*, 2012, pp. 85–95.

[10] F. Zeng, "Design and implementation of a QoS system based on OpenFlow," in *Proc. IEEE ASID*, 2013.

[11] Valve Corporation, "Source multiplayer networking," Developer Documentation, 2020. [Online]. Available: https://developer.valvesoftware.com/wiki/Source_Multiplayer_Networking

[12] ITU-T Recommendation G.107, "The E-model: A computational model for use in transmission planning," ITU, 2015.

[13] P. Bosshart *et al.*, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM CCR*, vol. 44, no. 3, pp. 87–95, 2014.